Model 2: Flood Scene Segmentation on FloodNet

U-Net with scSE attention and a Dice + Focal loss, compared with three baselines and with the FloodNet paper

Project report and explainer

Flood major project, October 2026

> **Result in one line.** Under identical training conditions our model reaches **75.67 mIoU** on the FloodNet test split, **+3.95** over plain U-Net and **+2.23** over the best baseline (DeepLabV3+). On the four hardest classes it scores **67.70**, which is also **above the FloodNet paper's best model** (PSPNet-R101, 65.36), although its overall mIoU (79.69) remains higher than ours.

1. What Model 2 is

Our project analyses drone (UAV) images taken after a flood. Model 2 is the part that looks at a whole image and labels **every pixel** with what it shows: flooded building, dry building, flooded road, dry road, water, tree, vehicle, pool, grass or background. This task is called **semantic segmentation**. Its output is a colour mask the same size as the photo.

Model 2 has two jobs:

- **In the main pipeline** it is the flood-severity model. From the mask we can measure, for example, what share of buildings and roads are under water.
- **As a research contribution** it is the subject of a separate paper: a modified U-Net that does better on the classes that matter most for rescue and that standard models get wrong.

Segmentation is different from the detection done by Model 1 (YOLO on VisDrone). Detection draws a box around each person or vehicle. Segmentation does not count objects. It says *what each pixel is*, which is what you need to measure flooded area.

1.1 The work in steps

1. **Prepare data once.** Download the official FloodNet release (13 GB), check that every image has a mask, and resize to 1024 x 768 (notebook `Model2_FloodNet_1_Prep_Colab`).
2. **Train models under one protocol.** Each teammate trained two models on a Colab T4 GPU with exactly the same settings (notebooks `..._Teammate1` and `..._Teammate2`).
3. **Evaluate once on the test split.** Each run keeps its best checkpoint by validation score and is then scored once on the 448 test images.
4. **Compare.** `scripts/floodnet_compare.py` builds the comparison table, the ablation table and the per-class chart.
5. **Ablation (in progress).** Two extra runs by teammate 3 separate the effect of the attention from the effect of the loss (Section 8).

2. Dataset: FloodNet

FloodNet-Supervised v1.0 (Rahnemoonfar et al., IEEE Access 2021) contains 2,343 high-resolution UAV images (about 4000 x 3000 pixels) taken after Hurricane Harvey in Texas, each with a pixel-level mask of 10 classes. It is released under the CDLA-Permissive 1.0 licence.

Split	Images	Used for
Train	1,445	Learning the weights
Validation	450	Choosing the best epoch (model selection)
Test	448	Final reported numbers only, used once

We use the **official split exactly as given**. Re-splitting would make our numbers impossible to compare with the paper. We did not use the EarthVision 2021 challenge version, which has only about 400 labelled images.

2.1 The 10 classes

ID	Class	Notes
0	background	Anything not in the other classes; excluded from the headline mIoU
1	building-flooded	Hard class: looks like a normal roof, flooding is only visible at the edges
2	building-non-flooded	
3	road-flooded	Hard class: muddy water over asphalt, easily confused with water
4	road-non-flooded	
5	water	Flood water and natural water bodies
6	tree	
7	vehicle	Hard class: very small after resizing, rare
8	pool	Hard class: small and rare, can look like clean water
9	grass	Most common class

The classes are **very unbalanced**: grass, trees and water cover most of the pixels, while vehicles and pools cover a tiny fraction. A model can score well overall while almost ignoring the small classes. That is why we report per-class scores and a separate hard-class score.

2.2 Pre-processing

- Every image is resized once to **1024 x 768** (about 3.9 times smaller) and cached, so all models see identical input.
- Images use area interpolation; masks use nearest-neighbour, so no invented class values appear at edges.
- Training uses random **512 x 512 crops**; evaluation uses the **full 1024 x 768 frame**.
- The prep script stops with an error on a missing mask, unreadable file, size mismatch or a class id outside 0-9.

3. Research scope

3.1 The problem

In flood imagery the most useful classes for rescue (flooded buildings, flooded roads, vehicles, pools) are exactly the ones standard models handle worst. In the FloodNet paper even the best model reaches only 46.15 IoU on vehicles and 64.19 on pools, against over 89 on trees and grass. Two reasons are visible in the data: these classes are **rare** (few pixels, so standard cross-entropy loss pays them little attention) and they are **hard to tell apart** from neighbours (flooded road vs. water, flooded vs. dry roof).

3.2 Research question

> Can a standard U-Net, given (a) attention that re-weights its features and (b) a loss that is not dominated by frequent classes, segment the hard flood classes better than established models trained under the same conditions?

3.3 What we propose

- **scSE attention** (concurrent spatial and channel squeeze-and-excitation, Roy et al., MICCAI 2018) in every U-Net decoder block. It learns *which feature channels* matter and *where in the image* to focus.
- **Dice + Focal hybrid loss** instead of cross-entropy. Dice rewards overlap per class regardless of class size; Focal concentrates on pixels the model still gets wrong.
- Same ResNet-50 encoder as all baselines, so any gain comes from these two changes, **not from a bigger network**.

3.4 Contributions

1. A **controlled comparison** of U-Net, DeepLabV3+ and PSPNet on the official FloodNet split with one identical protocol (the paper's own models were not trained identically).
2. A proposed model that improves mean IoU by **+3.95** over its plain version and the hard-class score by **+6.28**.
3. An **ablation** separating attention from loss (two runs in progress).
4. An honest comparison with the published FloodNet numbers, including where we are better and where we are not.

3.5 What is outside the scope

- Beating the paper's overall 79.69 mIoU. Our setup is deliberately lighter (ResNet-50, downsampled images).
- Full-resolution (4000 x 3000) training. Downsampling makes vehicles tiny; tiled full-resolution training is future work.
- Semi-supervised learning on the unlabelled FloodNet images, and other datasets.

3.6 The claim we can defend

> "Our model improves over baselines we trained under identical conditions, especially on the hard classes, and exceeds the published FloodNet results on the hard classes while using a smaller backbone." We do **not** claim to beat the paper's overall mIoU. Before writing "no prior work does X" we will run a literature search and say "to the best of our knowledge".

4. Method

4.1 Models compared

All four use a **ResNet-50 encoder** pre-trained on ImageNet. The **encoder** turns the image into compact features; the **decoder** turns those features back into a full-size mask. The models differ only in the decoder, the attention and the loss.

Model	Idea in one line	Role
U-Net	Encoder shrinks the image to learn 'what', decoder grows it back; skip connections pass fine detail across	Baseline, and the plain version of ours
DeepLabV3+	Atrous (dilated) convolutions at several rates see context at many scales; light decoder sharpens edges	Baseline (also in the FloodNet paper)
PSPNet	Pyramid pooling summarises the whole image at 4 grid sizes to add global context	Baseline (best model in the FloodNet paper)
Ours	**U-Net + scSE attention in every decoder block + Dice/Focal loss**	**Proposed model**

4.2 scSE attention

scSE combines two small blocks and adds their outputs:

- **cSE (channel squeeze-and-excitation):** averages each feature channel over the whole image, passes the averages through a tiny network, and multiplies every channel by a learned weight between 0 and 1. Useful channels are boosted, useless ones are damped.
- **sSE (spatial squeeze-and-excitation):** a 1 x 1 convolution produces one weight per pixel, so the model can boost locations that matter (for example the edge of a flooded roof).

It adds only about **1.3 M parameters** (32.52 M to 33.82 M) but costs speed, because it runs at every decoder resolution (Section 7.3).

4.3 Loss functions

The **loss** is a single number that says how wrong the prediction is; training adjusts the weights to make it smaller.

- **Cross-entropy (CE)**, used by the baselines: averages, over all pixels, how unlikely the model found the correct class. Every pixel counts equally, so a class with 1 % of pixels has almost no influence.

$$\mathrm{CE} = -(1/N) * \Sigma \log p(\text{correct class})$$

- **Dice loss**: 1 minus the Dice overlap, computed per class and then averaged over classes. A tiny class counts as much as a huge one.

$$\mathrm{Dice} = 2\cdot|\mathrm{Prediction} \cap \mathrm{Truth}| \,/\, (|\mathrm{Prediction}| + |\mathrm{Truth}|), \quad \text{Dice loss} = 1 - \mathrm{Dice}$$

- **Focal loss**: cross-entropy multiplied by (1 − p)^γ (γ = 2). Pixels the model already gets right (p close to 1) are almost ignored, so learning focuses on hard pixels.

$$\mathrm{Focal} = -(1-p)^{\gamma} \cdot \log p$$

Our **hybrid loss** is simply Dice + Focal, with equal weight.

4.4 Training protocol (identical for every run)

Setting	Value	What it means
Optimiser	AdamW, lr 2e-4, weight decay 1e-4	The rule that updates the weights after each batch; weight decay keeps weights small to reduce overfitting
Learning-rate schedule	1 epoch linear warm-up, then cosine decay	Start gently, then lower the step size smoothly towards zero so the model settles
Epochs / batch size	80 epochs, batch 8	80 full passes over the 1,445 training images, 8 crops per step
Augmentation	Scale +/-25 %, 512 crop, h/v flip, 90 deg rotation, brightness/contrast	Random changes to training images so the model generalises (drone images have no fixed 'up')
Precision	AMP (fp16)	Half-precision maths on the GPU: faster, less memory, same accuracy
Encoder initialisation	ImageNet pre-trained	The ResNet-50 starts from weights already trained on 1.2 M everyday photos
Model selection	Best validation mIoU (9), checked every 2 epochs	We keep the checkpoint that scores best on the validation split
Reported numbers	Test split, evaluated once	The test split is never used for any decision, so the result is unbiased
Seed	0	Fixes randomness so the run is repeatable
Hardware	NVIDIA Tesla T4 (Google Colab)	Same GPU type for every run, so FPS values are comparable

5. Evaluation terms explained

This section explains every number in the results tables, from the basic building blocks to the headline metric.

5.1 Building blocks

Term	Meaning
Pixel	One point of the image. A 1024 x 768 test image has 786,432 pixels, each with one true class and one predicted class.
Ground truth (GT)	The correct mask drawn by human annotators.
Prediction	The class the model gives each pixel (the class with the highest probability).
True positive (TP)	For class c: pixels that are c and were predicted c.
False positive (FP)	Pixels predicted c that are actually something else (false alarm).
False negative (FN)	Pixels that are c but were predicted as something else (miss).
True negative (TN)	Pixels that are not c and were not predicted c. Not used by IoU.
Confusion matrix	A 10 x 10 table: row = true class, column = predicted class, value = number of pixels, summed over the whole test set. Every metric below is computed from it.

5.2 Pixel accuracy

$$\text{Pixel accuracy} = \text{correctly labelled pixels} \,/\, \text{all pixels}$$

Easy to understand, but **misleading on unbalanced data**. If grass, trees and water cover most of the image, a model that is good on those and poor on vehicles still gets about 90 %. In our results all four models are between 89.40 % and 90.98 %, even though their vehicle IoU ranges from 43.60 to 65.14. Accuracy hides exactly the difference we care about, so it is reported but **not used to rank models**.

5.3 IoU (Intersection over Union), also called Jaccard index

$$\text{IoU}(c) = \text{TP} \,/\, (\text{TP} + \text{FP} + \text{FN})$$

The overlap between the predicted region and the true region of a class, divided by their combined area. 100 % means a perfect match; 0 % means no overlap. It punishes both misses (FN) and false alarms (FP), and it is computed **per class**, so a small class is judged on its own pixels only.

Example: if the true vehicle area is 100 pixels and the model marks 80 pixels, 60 of them correct, then TP = 60, FP = 20, FN = 40 and IoU = 60 / 120 = **50 %**. Pixel accuracy on the same image could still be 99 %.

5.4 mIoU (mean IoU)

- **mIoU (9)**: average IoU over the 9 real classes, **excluding background**. This is the definition used in the FloodNet paper's Table 4 (PSPNet: 79.69 is the mean of its 9 per-class values), so it is our **headline metric**.
- **mIoU (10)**: average over all 10 classes including background. Background is a mixed 'everything else' class and is hard to score well (6.55 to 44.12 IoU in our runs), so this value is lower and noisier.

Because every class has the same weight in the mean, mIoU rewards models that do well on **all** classes, including the rare ones.

5.5 Hard-4

$$\text{Hard-4} = \text{mean IoU of Building-flooded, Road-flooded, Vehicle, Pool}$$

Our focus metric. These four classes are the lowest-scoring in every published FloodNet study and the most important for rescue planning. Our research claim is about this number.

5.6 Dice coefficient / F1 score

Dice = 2·TP / (2·TP + FP + FN). It measures the same overlap as IoU on a slightly different scale (Dice is always ≥ IoU; IoU 50 % equals Dice 66.7 %). For segmentation Dice and the pixel-level F1 score are identical. We use Dice as a **loss**, but report IoU, because the FloodNet paper does.

5.7 Precision and recall

- **Precision** = TP / (TP + FP): of the pixels we called flooded road, how many really are.
- **Recall** = TP / (TP + FN): of the real flooded-road pixels, how many we found.

IoU combines both: it is low if either is low. For the detection side of the project (Model 1) recall is the key number; see `docs/EVALUATION_METRICS.md`.

5.8 Efficiency metrics

- **Params (M)**: millions of learnable weights. Indicates model size and memory.
- **FPS** (frames per second): how many 1024 x 768 images the model processes per second on one GPU (batch 1, fp16, T4). Higher is faster. FPS depends on the GPU, so only values measured on the same GPU type can be compared; all of ours are on a Tesla T4.
- **Train h**: wall-clock training time for 80 epochs on the T4.

5.9 Training terms

Term	Meaning
Epoch	One pass through all 1,445 training images.
Batch	The group of images (8 crops) processed before each weight update.
Learning rate (lr)	How large each weight update is (2e-4 here).
Overfitting	Memorising training images instead of learning general patterns; seen as validation score falling while training loss keeps dropping.
Validation split	Images used during training only to pick the best epoch.
Test split	Images held back until the end and used once, for the reported numbers.
Best epoch	The epoch whose checkpoint had the highest validation mIoU (9); that checkpoint is the one tested.
Checkpoint	A saved copy of the model weights (best.pt).
Pre-training (ImageNet)	Starting the encoder from weights learned on 1.2 M everyday photos, so it already recognises edges, textures and shapes.
Ablation	Removing or adding one component at a time to measure its individual effect.
Seed	The starting value for randomness. Repeating with seeds 1 and 2 shows how much results vary by chance.

6. Results

All models below were trained by us on the official split with the identical protocol of Section 4.4, and evaluated once on the 448 test images.

6.1 Overall comparison

Model	mIoU (9)	mIoU (10)	Pixel acc.	Hard-4	Params (M)	FPS	Best epoch	Train h
U-Net (R50)	71.72	65.20	89.40	61.42	32.52	24.7	54	1.73
DeepLabV3+ (R50)	73.44	70.51	90.77	62.97	26.68	34.6	76	1.56
PSPNet (R50)	69.57	65.62	89.99	56.74	24.34	62.6	66	1.32
Ours: U-Net (R50) + scSE + Dice/Focal	**75.67**	**72.36**	**90.98**	**67.70**	**33.82**	**17.9**	**76**	**2.48**

Table 1. Test results. Green row = our model. mIoU, accuracy and Hard-4 in %. FPS: batch 1, 1024 x 768, fp16, Tesla T4.

Main observations

- Ours is best on every quality metric: mIoU (9) **75.67**, mIoU (10) **72.36**, pixel accuracy **90.98**, Hard-4 **67.70**.
- Against plain U-Net (its own baseline) ours gains **+3.95** mIoU (9) and **+6.28** Hard-4, with only 1.3 M more parameters.
- Against the strongest baseline, DeepLabV3+, ours gains **+2.23** mIoU (9) and **+4.73** Hard-4.
- Pixel accuracy barely moves (89.40 to 90.98) while Hard-4 moves 10.96 points (56.74 to 67.70). This shows why accuracy is not used to rank models.
- The cost is speed: ours runs at 17.9 FPS, against 24.7 for U-Net and 62.6 for PSPNet. 17.9 FPS is still well above what offline drone-image analysis needs.

6.2 Per-class IoU, with the FloodNet paper

Model	Bld-F	Bld-NF	Road-F	Road-NF	Water	Tree	Vehicle	Pool	Grass	mIoU (9)	Hard-4
ENet [paper] *	6.94	47.35	12.49	48.43	48.95	68.36	32.26	42.49	76.23	42.61	23.55
DeepLabV3+ [paper] *	32.70	72.80	52.00	70.20	75.20	77.00	42.50	47.10	84.30	61.53	43.58
PSPNet-R101 [paper] *	68.93	89.75	82.16	91.18	92.00	89.55	46.15	64.19	93.29	79.69	65.36
U-Net (R50)	73.70	79.49	54.65	82.14	69.46	80.97	55.36	61.98	87.70	71.72	61.42
DeepLabV3+ (R50)	73.60	80.12	63.46	83.18	75.68	81.39	53.54	61.28	88.69	73.44	62.97
PSPNet (R50)	69.99	75.35	57.91	78.92	74.88	81.89	43.60	55.47	88.10	69.57	56.74
Ours (U-Net+scSE+D/F)	**73.67**	**80.06**	**63.96**	**83.01**	**75.66**	**82.60**	**65.14**	**68.02**	**88.90**	**75.67**	**67.70**

Table 2. Per-class test IoU (%). Grey rows (*) are published in FloodNet Table 4 and are context only: 713 x 713 crops, PSPNet with ResNet-101, a different training setup. Green = ours.

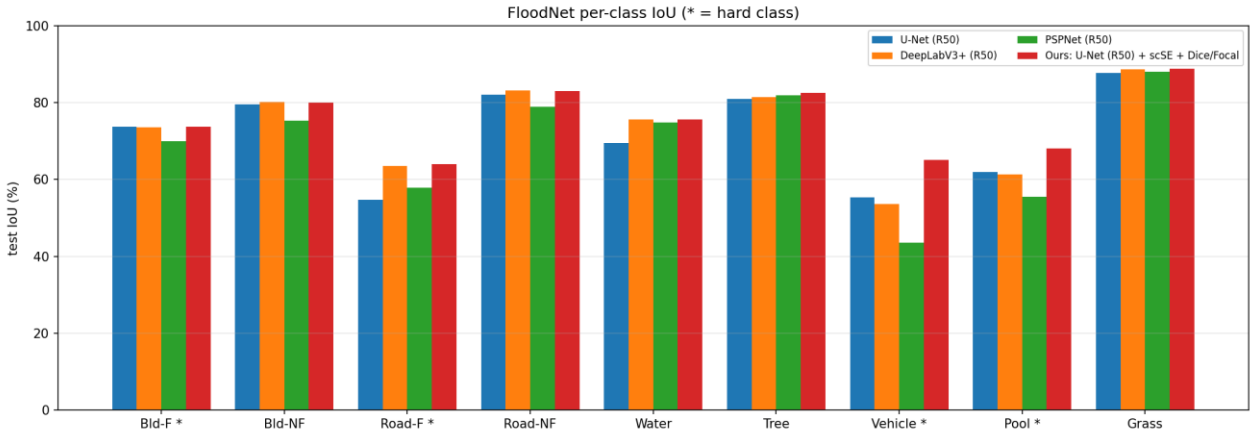


*Figure 1. Per-class test IoU of our four trained models. * = hard class. The largest gains of ours (red) are on Vehicle and Pool.*

Where the gain comes from. Ours is best or within 0.2 points of the best on every class, but most of its advantage sits in the rare classes: **Vehicle 65.14** (best baseline 55.36, +9.78) and **Pool 68.02** (best baseline 61.98, +6.04). On large common classes (grass, tree, dry road) all models are within about 2 points. This is the pattern we expect from a loss that weights classes equally and attention that sharpens small regions.

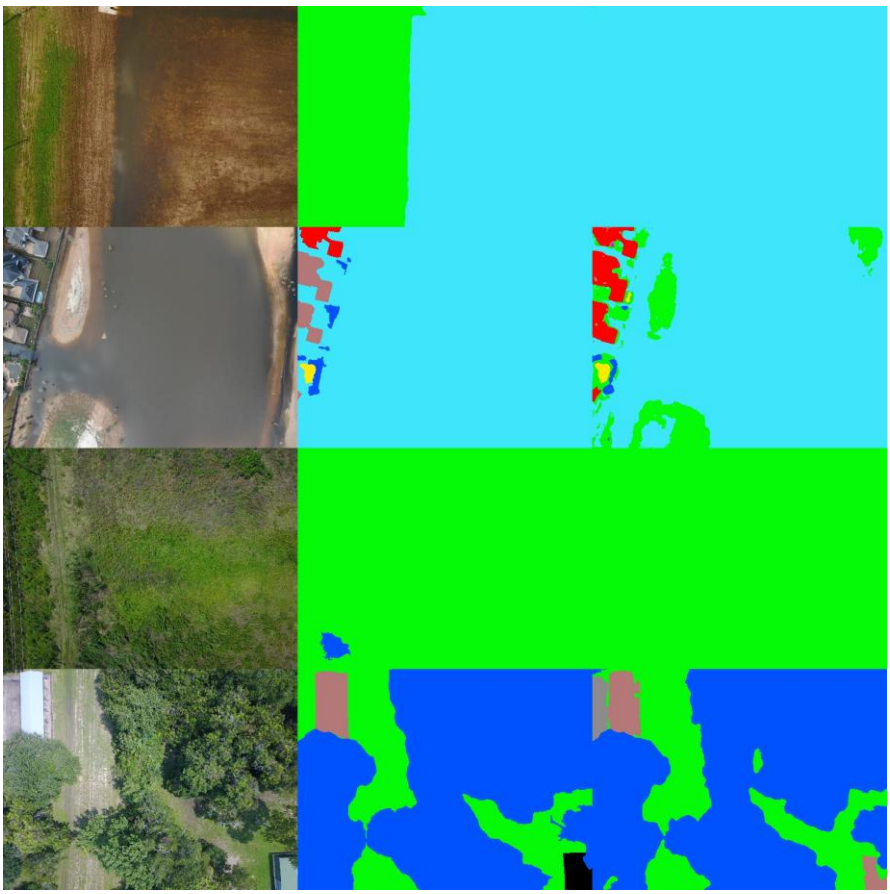


Figure 2. Our model on four test images. Left: photo. Middle: ground truth. Right: prediction. Colours: cyan water, green grass, blue tree, red flooded building, brown dry building, yellow pool.

Row 2 shows a typical remaining error: some flooded-building and grass pixels appear where the ground truth has dry buildings and water. Flooded vs. dry roofs are among the hardest decisions in the dataset.

7. Comparison with the baseline paper

The baseline paper is **FloodNet** (Rahnemoonfar et al., IEEE Access, vol. 9, 2021), which introduced the dataset and benchmarked ENet, DeepLabV3+ and PSPNet. Its best model is PSPNet with a ResNet-101 backbone at **79.69 mIoU (9)**.

7.1 The two setups are not identical

Aspect	FloodNet paper (baseline)	Our work
Backbone of best model	PSPNet with ResNet-101 (about twice as deep)	ResNet-50 for every model (~24-34 M total params)
Input size	713 x 713 crops	1024 x 768 frames, 512 x 512 training crops
Training schedule	Their own (not identical across models)	Identical for all models: 80 epochs, AdamW, cosine, seed 0
Data split	Official FloodNet split	Same official split (1445 / 450 / 448)
Metric	mIoU over 9 classes on test	Same definition (mIoU over 9 classes on test)
Purpose of the numbers	Benchmark of standard models	Controlled comparison + our proposed model

Because resolution, backbone and schedule differ, the paper's rows are shown as **context, not as a like-for-like comparison**. Our own four rows are the fair comparison.

7.2 Ours vs. the paper's best model (PSPNet-R101)

Class	PSPNet-R101 (paper)	Ours	Difference	Who is better
Bld-F (hard)	68.93	73.67	+4.74	**Ours**
Bld-NF	89.75	80.06	-9.69	Paper
Road-F (hard)	82.16	63.96	-18.20	Paper
Road-NF	91.18	83.01	-8.17	Paper
Water	92.00	75.66	-16.34	Paper
Tree	89.55	82.60	-6.95	Paper
Vehicle (hard)	46.15	65.14	+18.99	**Ours**
Pool (hard)	64.19	68.02	+3.83	**Ours**
Grass	93.29	88.90	-4.39	Paper
mIoU (9)	**79.69**	**75.67**	**-4.02**	**Paper**
Hard-4	**65.36**	**67.70**	**+2.34**	**Ours**

Table 3. Our model against the paper's best model, per class. Positive difference = ours is higher.

What this shows

- **Overall mIoU: the paper is higher** (79.69 vs. 75.67, −4.02). It uses a much deeper backbone (ResNet-101) and larger crops; we do not claim to beat it overall.
- **Hard classes: ours is higher** (Hard-4 67.70 vs. 65.36, +2.34). Vehicle improves by **+18.99** (46.15 to 65.14), flooded building by +4.74 and pool by +3.83.
- The paper's model is better on large classes (water, roads, trees, grass). The biggest gaps are flooded road (−18.20) and water (−16.34), two classes that are easily confused with each other. Larger context (bigger crops, deeper backbone) probably helps there.

- Against the paper's **DeepLabV3+** (61.53), our model is **+14.14** higher, and even our own DeepLabV3+ baseline (73.44) beats it by 11.91. The paper's training of that model was clearly weaker, which is one more reason to compare models only under one protocol.

7.3 Accuracy vs. speed

Model	Params (M)	FPS (T4)	mIoU (9)	Hard-4
U-Net (R50)	32.52	24.7	71.72	61.42
DeepLabV3+ (R50)	26.68	34.6	73.44	62.97
PSPNet (R50)	24.34	62.6	69.57	56.74
Ours: U-Net (R50) + scSE + Dice/Focal	**33.82**	**17.9**	**75.67**	**67.70**

PSPNet is the fastest (62.6 FPS) but the least accurate. Ours is the most accurate but the slowest (17.9 FPS), because scSE runs at every decoder level on large feature maps. The paper does not report speed, so this trade-off is new information.

8. Ablation study

The ablation asks: of the +3.95 gain, how much comes from **scSE attention** and how much from the **Dice/Focal loss**? Two more runs answer this, each changing exactly one thing from plain U-Net. Teammate 3 runs them with `Model2_FloodNet_2_Train_Teammate3_Colab.ipynb`.

Step	mIoU (9)	Change	Hard-4	Change	Vehicle	Pool	Status
U-Net R50 (cross-entropy)	71.72	0.00	61.42	0.00	55.36	61.98	Done
+ scSE only (cross-entropy)	-	-	-	-	-	-	Teammate 3
+ Dice/Focal only (no scSE)	-	-	-	-	-	-	Teammate 3
+ scSE + Dice/Focal (ours)	**75.67**	**+3.95**	**67.70**	**+6.28**	**65.14**	**68.02**	**Done**

Table 4. Ablation ladder (test, %). Rows marked Teammate 3 are being trained; the table fills in automatically when their result files are added and `floodnet_compare.py` is re-run.

How to read the finished table:

- If **scSE only** captures most of the gain, the attention is the main contribution.
- If **Dice/Focal only** captures most of it, the loss is the main contribution (expected especially for Vehicle and Pool, the rarest classes).
- If each alone gives less than half and both together give +3.95, the two changes **complement** each other, which is the strongest result for the paper.

9. Limitations and future work

- **Resolution.** Images are downsampled about 3.9 times (4000 to 1024 pixels wide), so vehicles become a few pixels across. This is the main reason vehicle IoU is still below the other classes. Future work: tiled full-resolution training.
- **Label noise.** Other work (LightFormer, arXiv 2504.10834) reports missed or mislabelled vehicles in FloodNet annotations, which caps the achievable vehicle IoU.
- **Single seed.** Each model was trained once (seed 0). Repeats with seeds 1 and 2 would give a mean and spread, showing the gain is not luck.
- **Backbone.** We used ResNet-50 for a fair, light comparison. A ResNet-101 or transformer encoder would likely raise all numbers, including ours.
- **Speed.** scSE makes the model about 28 % slower than plain U-Net. A lighter attention placement (only on deeper decoder blocks) could recover speed.

10. Use in the main pipeline

The best checkpoint (`outputs/model2/runs/unet_r50_scse_hybrid/best.pt`) becomes the flood-severity model. It stores the run name, configuration, class list and frame size, so it can be rebuilt with `floodnet_train.build_model(config, weights=None)`. From its masks the pipeline can compute, per image, the share of flooded buildings, flooded roads and water, which feeds the severity assessment.

11. Files and reproduction

File	Role
notebooks/Model2_FloodNet_1_Prep_Colab.ipynb	Download, verify, resize (run once)
notebooks/Model2_FloodNet_2_Train_Teammate{1,2,3}_Colab.ipynb	Training, 2 models per teammate
scripts/floodnet_prep.py	Pairing, validation, resizing, class statistics
scripts/floodnet_train.py	All runs, metrics, FPS, results JSON
scripts/floodnet_compare.py	Comparison table, ablation table, per-class chart
outputs/model2/results/*.json	Final numbers of every run
outputs/model2/comparison.md, ablation.md, per_class.png	Generated tables and figure

12. References

[1] M. Rahnemoonfar, T. Chowdhury, A. Sarkar, D. Varshney, M. Yari, and R. R. Murphy, "FloodNet: A High Resolution Aerial Imagery Dataset for Post Flood Scene Understanding," *IEEE Access*, vol. 9, pp. 89644-89654, 2021.

[2] A. G. Roy, N. Navab, and C. Wachinger, "Concurrent Spatial and Channel 'Squeeze & Excitation' in Fully Convolutional Networks," *MICCAI*, 2018.

[3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, 2015.

[4] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation" (DeepLabV3+), *ECCV*, 2018.

[5] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid Scene Parsing Network" (PSPNet), *CVPR*, 2017.

[6] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," *ICCV*, 2017.

[7] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation" (Dice loss), *3DV*, 2016.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition" (ResNet), *CVPR*, 2016.